LAB 07

Signal Stage: a Library of Trend, Mean-Reversion, Momentum and Breakout Signals

Item	Detail
Topic	Topic 2: Technical Indicators, Trading Strategies and Metrics with TradingView and Alpaca
Pipeline stage	Signals
TSC reference	A2, K3, K7
Duration	25 min
Folder	labs/Lab07-python-indicators/
Files	Lab07-python-indicators/signals_stage.py, pipeline/signals.py, Lab07-python-indicators/solution/signals.py
Mode	Runs ONLINE with your Alpaca paper keys in labs/.env, or OFFLINE on the bundled data and the paper-broker simulator. Paper trading only.

Lion City Capital and every scenario in these materials are fictitious and for training only. All trading in this course is paper (simulated) trading. Nothing in these materials is investment advice or a recommendation to trade any security.

Scenario

Lion City Capital wants every strategy - from a TradingView prototype to an ML model - to produce the same thing: a target position per symbol, decided at the close and traded on the next bar. The quant team builds one signal library that the evaluation, search, execution, automation and diagnostic stages all call.

Objective

Build pipeline/signals.py with trend, mean-reversion, momentum and breakout signals that return target positions with next-bar alignment, a registry, an automated look-ahead test and a universe target file for the execution stage.

Before You Start

- Open a terminal in the labs folder and activate your virtual environment.
- Have your AI coding assistant open (for example Claude Code) in the labs folder if you want to rebuild the code from the prompts; the finished reference code is already in this folder.
- Record every result you are asked to check - the same evidence is asked for in the Practical Performance assessment.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Step-by-Step Instructions

Step 1: Agree the signal contract (A2)

Read the docstring of the reference Lab07-python-indicators/solution/signals.py. Every signal returns a float Series of target positions (1 = long, 0 = flat) decided with information up to the CLOSE of bar t; the position is HELD from bar t+1. Warm-up bars (NaN indicators) are flat.

Check: You can explain why a signal computed at Friday's close can only earn Monday's return.

Step 2: Build the trend signals (A2, K3)

Ask your assistant for the trend part of pipeline/signals.py.

AI prompt - paste into your AI coding assistant
Create labs/pipeline/signals.py. Helper _flat_until_ready(cond, *inputs) returns 1.0 where cond is true and 0.0 otherwise or while any input is NaN. Trend signals using pipeline.indicators: ema_cross(bars, fast=10, slow=30) long while ta_ema(fast) > ta_ema(slow); sma_cross(bars, fast=50, slow=200); trend_filter(bars, window=200) long while close > ta_sma(window). Each returns a float Series aligned to bars.index, using data up to bar t only. Rules: run from the labs folder; reuse labs/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: ema_cross(load_bars('SPY','2018-01-01','2025-12-31')).iloc[-1] is 1.0.

Step 3: Build mean-reversion, momentum and breakout signals (A2, K3)

Ask your assistant for the remaining families.

AI prompt - paste into your AI coding assistant
In labs/pipeline/signals.py add enter_exit(entry, exit_) - a stateful long/flat series that is 1 from an entry bar until the next exit bar (entry wins a tie, NaN treated as False); rsi_reversion(bars, n=2, entry=10, exit=70) = enter_exit(RSI < entry, RSI > exit); dip_buy(bars, n_down=3, hold=3) = long for `hold` bars after a bar completing >= n_down consecutive down closes; momentum(bars, lookback=126, threshold=0.0) long while the past lookback-bar return > threshold; donchian_breakout(bars, entry=55, exit=20) buy a close above the PRIOR entry-bar high (rolling max shifted 1), sell a close below the prior exit-bar low; bollinger_breakout(bars, window=20, k=2.0) buy a close above the upper band, sell below the middle band. Rules: run from the labs folder; reuse labs/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: Each function returns only 0.0 and 1.0 values for SPY.

Step 4: Add the registry, next-bar alignment and the look-ahead test (A2, K3)

Ask your assistant for the shared helpers.

AI prompt - paste into your AI coding assistant
In labs/pipeline/signals.py add SIGNALS = {name: (function, family, default_params)} for the eight signals; target_position(name, bars, **params) (params override defaults, Series renamed to name); next_bar_position(target) = target.shift(1).fillna(0) (the position actually held); lookahead_test(name, bars, checks=25, seed=0, fn=None, **params): pick 25 random dates t in the last two-thirds of the sample and recompute the signal (a) on bars.loc[:t] only and (b) after multiplying every price AFTER t by 1.5 and by 0.5; count dates where any value up to t differs from the full-sample signal (any difference = look-ahead); describe_signal(target) with latest_target, entries, exposure, avg_hold_bars and last_change; latest_targets(symbols, name, params, start, end) returning a DataFrame of symbol, as_of, close, target, last_change. Rules: run from the labs folder; reuse labs/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.

Check: lookahead_test('ema_cross', bars) returns 0.

Step 5: Build the signals command (A2)

Ask your assistant for run() and main().

AI prompt - paste into your AI coding assistant
In labs/pipeline/signals.py add run(symbol='SPY') for `python -m pipeline signals --symbol`: for every signal in SIGNALS print family, params, latest target, entries, exposure, average holding bars, last change date and the look-ahead result (PASS or FAIL (n)); print one example of next-bar alignment for ema_cross (the close where the target changed and the first bar it is held); compute latest_targets for config universe.trade with config strategy name/params, print it and save _outputs/Lab07_targets.csv (input to the execution stage); save chart Lab07_signals.png (SPY 2024-2025 with green shading where ema_cross, rsi_reversion, momentum and donchian_breakout are long). Exit code 1 if any look-ahead test fails. Rules: run from the labs folder; reuse labs/common (config, data, broker, indicators, metrics, backtest, plotting) and earlier pipeline stages instead of rewriting them; do not modify common/ or data/; read every setting from pipeline/config.yaml via pipeline.config.load_config(); never put API keys or secrets in code; paper trading only; keep a main(argv) function so `python -m pipeline <command>` works; add short comments a finance professional can follow.

Check: python -m pipeline signals --help shows --symbol.

Step 6: Run the signal stage (A2, K3)

Run: python -m pipeline signals --symbol SPY (same as: python3 Lab07-python-indicators/signals_stage.py).

Check: ema_cross 26 entries, 71% exposure, 54.7 bars average hold, last change 2025-11-28; sma_cross 5 entries; trend_filter 24; rsi_reversion 88 entries, 20% exposure, 4.6 bars; dip_buy 102 entries, 19%, 3.7 bars; momentum 33; donchian_breakout 20; bollinger_breakout 37 (latest target 0); all look-ahead PASS. Alignment: target changes to 1 at the close of 2025-11-28 and is held from 2025-12-01. Universe: 7 of 9 symbols long (MSFT and TLT flat); _outputs/Lab07_targets.csv and Lab07_signals.png.

Step 7: Catch a look-ahead bug on purpose (K3)

Temporarily change ema_cross to use bars['close'].shift(-1) (tomorrow's close) and rerun the stage. Then undo the change and rerun.

Check: With the bug ema_cross shows 'FAIL (25)' (the future-shock test changes its history every time) and the command exits with code 1; after undoing it all signals PASS again.

Step 8: Relate signal frequency to trading cost (K7)

Compare entries per signal: dip_buy (102) and rsi_reversion (88) against sma_cross (5).

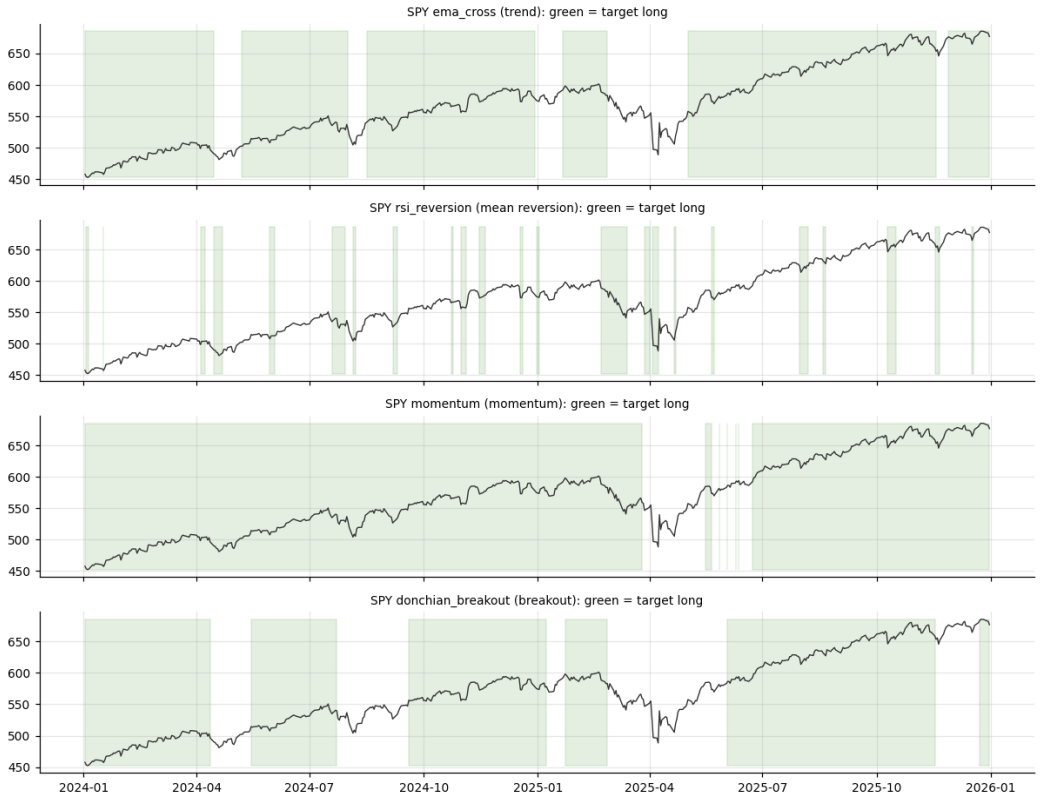
AI prompt - paste into your AI coding assistant
Using the signal table from python -m pipeline signals, estimate for each signal the round trips per year over 2018-2025 and the annual cost drag at 3 bps per round trip on USD 100,000, assuming the full equity is traded. Rank the signals from most to least cost-sensitive and say which ones need a strong edge to survive costs.

Check: dip_buy and rsi_reversion trade about 11-13 round trips a year versus under one for sma_cross, so they are the most cost-sensitive.

Expected Output

python -m pipeline signals --symbol SPY: eight signals in four families, all pass the look-ahead truncation test; ema_cross 26 entries and 71% exposure (latest flip at the close of 2025-11-28, held from 2025-12-01); dip_buy 102 entries; universe targets 7 of 9 long (MSFT, TLT flat) saved to _outputs/Lab07_targets.csv; chart _outputs/Lab07_signals.png.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.



Lab 7 output (Lab07_signals.png), offline mode on the bundled data.

Debrief

A single signal contract - target positions decided at the close, held from the next bar - is what lets one library feed backtests, live execution and diagnostics (A2). Indicators (K3) become signals through simple, testable rules, and the number of entries each rule produces is the first estimate of its trading cost (K7).

Troubleshooting

Symptom	What to do
Signals contain NaN	Wrap conditions with _flat_until_ready so warm-up bars are 0, not NaN.
A look-ahead test fails	Look for shift(-1), center=True or any use of data after bar t in that signal.
Targets file is empty	Run python -m pipeline ingest first or check universe.trade in config.yaml.
TypeError on a column called hist	Use df['hist'], not df.hist - hist is also a pandas method.

© Tertiary Infotech Academy Pte Ltd. All rights reserved. This document is provided for the explicit use and guidance of parties approved by Tertiary Infotech Academy Pte Ltd as an information resource only. Any other use, including reproduction, publication, distribution or storage in a retrieval system in any form, without the express permission of Tertiary Infotech Academy Pte Ltd, is strictly prohibited.